electronics

MDPI

Article

Numerical Solution of the Poisson Equation Using Finite Difference Matrix Operators

Mohammad Asif Zaman

Department of Electrical Engineering, Stanford University, Stanford, CA 94305, USA; zaman@stanford.edu

Abstract: The Poisson equation frequently emerges in many fields of science and engineering. As exact solutions are rarely possible, numerical approaches are of great interest. Despite this, a succinct discussion of a systematic approach to constructing a flexible and general numerical Poisson solver can be difficult to find. In this introductory paper, a comprehensive discussion is presented on how to build a finite difference matrix solver that can solve the Poisson equation for arbitrary geometry and boundary conditions. The boundary conditions are implemented in a systematic way that enables easy modification of the solver for different problems. An image-based geometry-definition approach is also discussed. Python code of the numerical recipe is made publicly available. Numerical examples are presented that show how to set up the solver for different problems.

Keywords: Poisson equation; matrix operators; finite difference method; partial differential equations; tutorial

Citation: Zaman, M.A. Numerical Solution of the Poisson Equation Using Finite Difference Matrix Operators. *Electronics* **2022**, *11*, 2365. https://doi.org/10.3390/electronics11152365

Academic Editors: Flavio Canavero and Nikolay Hinov

Received: 21 June 2022
Accepted: 25 July 2022
Published: 28 July 2022

1. Introduction

The Poisson equation is an elliptical partial differential equation (PDE) and is ubiquitous in many areas of physics and engineering. Two of its common uses include modeling electrostatic potential and gravitational potential. Some forms of the Poisson equation also appear in fluid flow [1] and heat transfer problems [2,3]. Often, analytic solutions of such problems are not possible for cases of practical interest due to complex geometries. Numerically solving the Poisson equation is the only way to study these systems. As such, numerical solution methods for the Poisson equation are of significant interest [2,4,5].

There are several methods for solving the Poisson equation numerically [6]. The Finite-Difference Method (FDM) is one of the most simple and popular approaches [7–10]. This method involves replacing the continuous derivative operators with approximate, discrete finite-difference operators [11] that take the form of matrices. The method is intuitive and the solution process involves well-established linear algebraic methods. It suffers from some drawbacks, such as accurately representing complex geometries and boundaries. Methods such as the Finite Element Method (FEM) are better at handling arbitrary geometries. However, due to the ease of implementation and the intuitive nature of FDM, it remains a very popular approach for solving PDEs.

There are several references that discuss solving the Poisson equation using FDM [12]. While these papers cover many details, they usually do not present a systematic approach of constructing the numerical system (i.e., system matrix formation) for arbitrary geometries and arbitrary boundary conditions. Thus, for every variation of the Poisson equation, the linear algebraic system has to be reconstructed from scratch. The goal of this paper is to introduce a systematic approach for solving the Poisson equation for any arbitrary geometry and boundary conditions with minimal modification of the numerical solver/computer code. This would allow one to study a variety of configurations of the physical system without having to reformulate the numerical recipe.

Setting up an FDM solver has some basic steps. The first step is discretization of the solution space. A series of grid points in space where the equation will be solved is defined.

At each of these points, the Poisson equation is written in discrete form using difference equations. In the conventional approach, the boundary conditions are incorporated at this step by modifying the difference equations at boundary points. This step depends on the geometry and the type of the boundary conditions. After this adjustment, we are left with a system of linear equations from which a full rank matrix equation can be formed. This matrix is referred to as the *system matrix*. The system is then solved. Note that when the geometry or the boundary conditions are altered, the system matrix changes and thus has to be reconstructed. Most references do not provide the reader with a systematic approach to do this. In many cases, one is required to study different geometrical variations of the same problem. Thus, one is often left with the cumbersome task of modifying their numerical solver for each configuration.

In this paper, the system matrix is defined as the combination of two matrices. One matrix is independent of the problem geometry and thus remains unchanged for all cases. This is referred to as the *derivative operator matrix*. For the Poisson equation, this is the second-derivative operator. The other matrix is defined to be dependent on the geometry and the boundary conditions. This is referred to as the *boundary operator matrix*. Modifying the boundary operator matrix turns out to be relatively simple and intuitive. It is shown that the system matrix can be constructed by compiling rows from these matrices appropriately. Note that when the geometry and/or the boundary conditions change, only the boundary operator matrix and the system matrix construction procedure changes. Thus, different geometry variations (and/or boundary conditions) could be investigated with minor adjustments of the same numerical code. This is the key advantage of the proposed method.

In this paper, a complete numerical recipe of how to implement a flexible Poisson equation solver is presented in detail. In addition, a Python implementation of the algorithm is discussed and is made available to the readers through a GitHub repository [13]. Some example numerical results are also presented that showcase the capabilities of the solver.

The rest of the paper is organized as follows: Section 2 defines the Poisson equation and the types of boundary conditions typically encountered, Section 3 discusses the finite difference formulation for one-dimensional (1D) and two-dimensional (2D) cases; the computer implementation of the algorithm using Python is discussed in Section 4; Section 5 presents numerical results for a few example problems; and concluding remarks are made in Section 6.

2. The Poisson Equation and Boundary Conditions

The Poisson equation consists of three second-order spatial derivatives. The general form of the equation in Cartesian coordinates is:

$$\Delta u(x,y,z) = f(x,y,z)\,, \text{ on } \Omega\,. \tag{1}$$

where

$$\Delta = \nabla^2 = \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2} + \frac{\partial^2}{\partial z^2}\,, \tag{2}$$

is the Laplacian operator in Cartesian coordinates, x, y, z are the independent Cartesian spatial coordinates, and Ω is the solution domain (i.e., the set of x, y, z values for which the equation is to be solved). The dependent variable, u, is the function that is to be solved, and f is known as the *source function*, which is usually given. In general, it is a function of the space coordinates. In the special case of $f = 0$, the equation reduces to the Laplace equation.

Like any differential equation, boundary conditions must be defined before a solution can be attempted. A complete definition of the problem includes Equation (1) with a defined source function and boundary conditions enforced on u or its first derivatives (i.e., $\frac{\partial u}{\partial x}$, $\frac{\partial u}{\partial y}$, and $\frac{\partial u}{\partial z}$). Boundary conditions may also be enforced on both u and its derivatives simultaneously. The boundary of the solution domain Ω is denoted as $\partial\Omega$. The boundary

can be subdivided into M number of arbitrary segments or regions, $\partial\Omega_1, \partial\Omega_2, \cdots, \partial\Omega_M$, each having its own boundary conditions. Usually, four common types of boundary conditions are encountered:

- Dirichlet Boundary Conditions, having the form: $u|_{\partial\Omega_i} = u_i$
- Neumann Boundary Conditions, having the form: $\frac{\partial u}{\partial \eta}|_{\partial\Omega_i} = g_i$
- Mixed Boundary Conditions, having the form: $u|_{\partial\Omega_i} = u_i$, $\frac{\partial u}{\partial \eta}|_{\partial\Omega_j} = g_j$
- Robin Boundary Conditions, having the form: $\left(Au + B\frac{\partial u}{\partial \eta}\right)|_{\partial\Omega_i} = g_i$

Here $\eta \in \{x, y, z\}$ represents one of the Cartesian spatial variables, $i, j \in \{1, 2, \cdots, M\}$, A,B are constants, and u_i, g_j are given quantities. Note that for Neumann boundary conditions, since the constraint is applied on the derivative, u can only be determined up to a constant. Hence, an additional constraint is required to obtain a unique solution. This constraint can have the following form [14,15]:

$$\int_{\Omega} u dx = 0 \,. \tag{3}$$

The boundary regions can also be specified inside the simulation domain itself and not necessary defined on the outer perimeter (i.e., some $\partial\Omega_i$ among $\partial\Omega_1, \partial\Omega_2, \cdots, \partial\Omega_M$ may not necessarily reside on the outer edges of Ω). This is more common for Dirichlet boundaries [11]. The solution scheme presented here makes no distinction between the inner and outer boundary regions. The method works for any arbitrary $\partial\Omega_i$ for all i.

3. Finite Difference Discretization and Matrix Operators

All the quantities in Equation (1) are continuous variables. When solving numerically, the equation is solved only at discrete points. The first step is to convert the variables and the equation from continuous domain to discrete domain. The 1D case is considered first for simplicity. Higher-dimensional cases can be built from the one-dimensional case.

3.1. The Poisson Equation in One Dimension

Considering only one space variable, x, Equation (1) can be written as:

$$\frac{d^2u(x)}{dx^2} = f(x)\,, x \in \Omega\,. \tag{4}$$

where Ω is the solution domain defining the possible x values. A set of N discrete x values $x_1, x_2, \cdots, x_N$ of x is selected where the equation is to be solved. Then, the domain is $\Omega = [x_1, x_N]$. The boundary of the domain is assumed to be the outer edges: $\partial\Omega = \{x_1, x_N\}$. Some texts use the notation x_0 and x_{N+1} to indicate the boundary points and exclude them from the definition of the solution domain. However, in this paper, the edge points of the solution domain are taken as boundary points unless otherwise stated.

The domain points are referred to as x_i, $i = 1, 2, \cdots, N$. For simplicity, it is assumed that these points are separated by a constant number Δx (i.e., $x_{i+1} - x_i = \Delta x, \forall i$). The corresponding values of f and u are denoted f_i and u_i, respectively (i.e., $f(x_i) = f_i$, $u(x_i) = u_i$). It is convenient to express these sets of discrete points as vectors $\mathbf{x} = [x_1\ x_2\ \cdots\ x_N]^\mathsf{T}$, $\mathbf{f} = [f_1\ f_2\ \cdots\ f_N]^\mathsf{T}$, and $\mathbf{u} = [u_1\ u_2\ \cdots\ u_N]^\mathsf{T}$. Note that transpose notations were used to write the column vectors in a compact manner. With vector notation, the differential equation can be imagined as a linear algebra problem where the vector $\mathbf{u}$ is to be solved for a given $\mathbf{f}$. The next step is to convert the second-derivative operator into a matrix operator so that a complete linear algebraic system can be formed.

To convert a continuous derivative operator to a discrete matrix operator, the starting point is the finite difference formulas for calculating the derivatives. The vectors $\mathbf{u}' = [u_1'\ u_2'\ \cdots\ u_N']^\mathsf{T}$ and $\mathbf{u}'' = [u_1''\ u_2''\ \cdots\ u_N'']^\mathsf{T}$ are defined to denote the first and second derivatives, respectively, corresponding to the points $\mathbf{x}$ (i.e., $u_i' = u'(x_i)$, and $u_i'' = u''(x_i)$).

Using the second-order center-difference formulas, the first and second derivatives can be expressed as [16]:

$$\left.\frac{du}{dx}\right|_{x=x_i} \approx u_i' = \frac{-u_{i-1}+u_{i+1}}{2\Delta x}, \tag{5}$$

$$\left.\frac{d^2u}{dx^2}\right|_{x=x_i} \approx u_i'' = \frac{u_{i-1}-2u_i+u_{i+1}}{(\Delta x)^2}. \tag{6}$$

Note that there are other higher-order finite difference formulas for the above derivatives. The widely used second-order center-difference formulas are selected for the current analysis. Now, from Equations (5) and (6), focusing only on three consecutive quantities u_{i-1}, u_i, u_{i+1}, the following matrix equation can be written:

$$u_i' = \frac{1}{2\Delta x}\begin{bmatrix}-1 & 0 & 1\end{bmatrix}\cdot\begin{bmatrix}u_{i-1}\\ u_i\\ u_{i+1}\end{bmatrix}, \tag{7}$$

$$u_i'' = \frac{1}{(\Delta x)^2}\begin{bmatrix}1 & -2 & 1\end{bmatrix}\cdot\begin{bmatrix}u_{i-1}\\ u_i\\ u_{i+1}\end{bmatrix}. \tag{8}$$

Equations (7) and (8) work fine for $i = 2$ to $N-1$. However, for the boundary points $i = 1$ and $i = N$, the subscripts overflow the bounds, requiring values of u_0 and u_{N+1}, respectively. Thus, the equation is not valid for those two cases (note that the outer boundary points are x_1 and x_N). We resort to using forward (for $i = 1$) and backward (for $i = N$) finite difference formulas for these two cases [16]:

$$u_1' = \frac{-3u_1+4u_2-u_3}{2\Delta x} = \frac{1}{2\Delta x}\begin{bmatrix}-3 & 4 & -1\end{bmatrix}\cdot\begin{bmatrix}u_1 & u_2 & u_3\end{bmatrix}^{\mathsf{T}}, \tag{9}$$

$$u_1'' = \frac{2u_1-5u_2+4u_3-u_4}{(\Delta x)^2} = \frac{1}{(\Delta x)^2}\begin{bmatrix}2 & -5 & 4 & -1\end{bmatrix}\cdot\begin{bmatrix}u_1 & u_2 & u_3 & u_4\end{bmatrix}^{\mathsf{T}}, \tag{10}$$

$$u_N' = \frac{u_{N-2}-4u_{N-1}+3u_N}{2\Delta x} = \frac{1}{2\Delta x}\begin{bmatrix}1 & -4 & 3\end{bmatrix}\cdot\begin{bmatrix}u_{N-2} & u_{N-1} & u_N\end{bmatrix}^{\mathsf{T}}, \tag{11}$$

$$u_N'' = \frac{-u_{N-3}+4u_{N-2}-5u_{N-1}+2u_N}{(\Delta x)^2} = \frac{1}{(\Delta x)^2}\begin{bmatrix}-1 & 4 & -5 & 2\end{bmatrix}\cdot\begin{bmatrix}u_{N-3} & u_{N-2} & u_{N-1} & u_N\end{bmatrix}^{\mathsf{T}}. \tag{12}$$

Now, using Equations (7) to (12), the first and second derivatives at all points in the solution space can be calculated. These equations for all i values can be compiled to form the following matrix equations:

$$\mathbf{u}' = \begin{bmatrix}u_1'\\ u_2'\\ u_3'\\ \vdots\\ u_{N-1}'\\ u_N'\end{bmatrix} = \underbrace{\frac{1}{2\Delta x}\begin{bmatrix}-3 & 4 & -1 & 0 & \cdots & 0 & 0 & 0 & 0\\ -1 & 0 & 1 & 0 & \cdots & 0 & 0 & 0 & 0\\ 0 & -1 & 0 & 1 & \cdots & 0 & 0 & 0 & 0\\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \vdots\\ 0 & 0 & 0 & 0 & \cdots & 0 & -1 & 0 & 1\\ 0 & 0 & 0 & 0 & \cdots & 0 & 1 & -4 & 3\end{bmatrix}}_{D_x}\cdot\begin{bmatrix}u_1\\ u_2\\ u_3\\ \vdots\\ u_{N-1}\\ u_N\end{bmatrix}, \tag{13}$$

$$\mathbf{u}'' = \begin{bmatrix} u_1'' \\ u_2'' \\ u_3'' \\ \vdots \\ u_{N-1}'' \\ u_N'' \end{bmatrix} = \underbrace{\frac{1}{(\Delta x)^2}\begin{bmatrix} 2 & -5 & 4 & -1 & \cdots & 0 & 0 & 0 & 0 \\ 1 & -2 & 1 & 0 & \cdots & 0 & 0 & 0 & 0 \\ 0 & 1 & -2 & 1 & \cdots & 0 & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & 0 & 1 & -2 & 1 \\ 0 & 0 & 0 & 0 & \cdots & -1 & 4 & -5 & 2 \end{bmatrix}}_{D_{2x}} \cdot \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ \vdots \\ u_{N-1} \\ u_N \end{bmatrix}. \tag{14}$$

It should be noted that for nonuniform grids, the term Δx would no longer be constant. As a result, in the place of the common Δx factor in the matrices, every matrix element would have a denominator corresponding to its own Δx value. Every other step discussed in this manuscript would remain unchanged. With the help of these matrices, the derivative operations can be expressed in linear algebra form as:

$$\mathbf{u}' = D_x\mathbf{u}\,, \tag{15}$$
$$\mathbf{u}'' = D_{2x}\mathbf{u}\,. \tag{16}$$

Here D_x and D_{2x} are $N \times N$ matrices, as shown in Equations (13) and (14). These are referred to as *matrix operators* for the first and second derivatives, respectively, as they operate on a vector to calculate its derivatives. Equation (4) can be converted to a discrete linear algebraic system using these matrix operators:

$$D_{2x}\mathbf{u} = \mathbf{f}\,, \text{ on } \Omega\,. \tag{17}$$

Now the boundary conditions must be implemented in such a way that they can be easily integrated with Equation (17).

Let us consider a general implementation of boundary conditions in matrix form:

$$B\mathbf{u} = \mathbf{g}\,, \text{ on } \partial\Omega\,. \tag{18}$$

Here B is referred to as the *boundary operator* matrix, and $\mathbf{g}$ is the known/given boundary function vector. For a given problem, the boundary operator matrix is determined by the type of the boundary conditions. For Dirichlet boundary conditions, the boundary function values are directly assigned to the boundary points. In such a case, B would be an identity matrix, and $\mathbf{g}$ would list the values of $\mathbf{u}$ at the boundary. For Neumann boundary conditions, the value of the first derivative is assigned at the boundary points. For this case, B would be the first-derivative matrix operator D_x, and $\mathbf{g}$ would list the values of $\mathbf{u}'$ at the boundary. Thus,

$$B = \begin{cases} I_N\,, & \text{for Dirichlet boundary conditions} \\ D_x\,, & \text{for Neumann boundary conditions} \end{cases} \tag{19}$$

Here I_N is the $N \times N$ identity matrix, and D_x is expressed in Equation (13). Note that although Equation (19) relates the quantities in such a way that B operates on the entire $\mathbf{u}$, the equation is valid only at the boundary $\partial\Omega$. In fact, $\mathbf{g}$ elements at non-boundary points are unknown. In other words, the *locations* of the boundaries are not defined explicitly within B. However, to understand which entries of B come from I_N and which entries come from D_x, information about the geometry of the system is required. In the following paragraphs, we will discuss how to compile the linear algebraic system so that only the relevant rows of B are used. This is where the remaining geometry information will be encoded. Before that, it is important to discuss how a multi-boundary system would be treated. As previously mentioned, $\partial\Omega$ can be subdivided into smaller regions

$\partial\Omega_1, \partial\Omega_2 \cdots \partial\Omega_M$ each with its own boundary conditions. Each $\partial\Omega_i$ corresponds to a set of specific rows in the boundary operator matrix. For such cases, $\mathbf{g}$ would be defined appropriately in a piecewise manner. For mixed boundary conditions, the rows of B would be a compilation of rows taken from either I_N or D_x depending on which boundary regime the corresponding equation falls under. It should also be noted that for Neumann boundary conditions, an additional constraint such as Equation (3) would be needed to obtain a unique solution.

After defining the matrix forms for the PDE and the boundary conditions, the process of integrating those into a single linear algebraic equation can be started. It is possible to simply solve Equation (17) under the constraint defined in Equation (19). However, an alternative and somewhat more elegant approach would be to form a single system matrix that incorporates both the PDE and the boundary conditions in a compact form. By observing the full form of the matrix and vectors in Equation (17), it can be noted that each row of the matrix defines an equation containing a few elements of $\mathbf{u}$. Since some of the elements of $\mathbf{u}$ are on the boundary and follow Equation (19), replacing the corresponding rows of D_{2x} with those of B can lead to the formation of a system matrix, $\mathcal{L}$. This can be mathematically described as:

$$\mathcal{L} = \begin{cases} B\,, & \text{on } \partial\Omega \\ D_{2x}\,, & \text{otherwise} \end{cases}. \tag{20}$$

Of course, similar changes to the right-hand vector would need to be made as well. The *system right-hand vector*, $\mathbf{b}$, can be defined as:

$$\mathbf{b} = \begin{cases} \mathbf{g}\,, & \text{on } \partial\Omega \\ \mathbf{f}\,, & \text{otherwise} \end{cases}. \tag{21}$$

According to this definition, if the k-th element of $\mathbf{u}$, u_k, corresponds to a Dirichlet boundary point, then the k-th row of $\mathcal{L}$ would be the k-th row of I_N. Further, the k-th element of $\mathbf{b}$ would be the k-th row of $\mathbf{g}$. All rows corresponding to non-boundary points will be identical to the rows of D_{2x}. Now, the system equation can finally be written as:

$$\mathcal{L}\mathbf{u} = \mathbf{b}\,. \tag{22}$$

Dimension-wise, $\mathcal{L}$ is an $N \times N$ matrix, and $\mathbf{b}$ is an $N \times 1$ vector. This is a fully defined linear system from which $\mathbf{u}$ can be easily solved. Note that the derivative operator, the boundary conditions, and the problem geometry are all encoded within $\mathcal{L}$ and b. Let us consider the 1D case with x_1 and x_N being the boundary points. For Dirichlet boundary conditions, this correspond to $[u_1\ u_N]^\mathsf{T} = [g_1\ g_N]^\mathsf{T}$, where g_1 and g_N are given. Using Equations (20) and (21), the matrix system is constructed and solved. Note that $g_i, \forall i \neq 1, N$, are not defined and are not needed to form $\mathbf{b}$.

One notable feature of the proposed system matrix construction procedure is that part of it is problem independent. Note that only the B matrix and the $\mathbf{g}$ vector depend on the boundary condition types at different boundaries. The D_{2x} matrix is the same for all problems. The construction of $\mathcal{L}$ involves swapping out certain rows of D_{2x} with B. This part depends on the problem geometry (as points on $\partial\Omega$ need to be identified). Thus, modifying the solver for different geometries and boundary conditions involves selecting the appropriate form of B and modifying the row-swapping operation of $\mathcal{L}$ only (the same operation must be done for the right-hand vector, also). This is a major benefit over conventional procedures for which the system matrix is constructed from scratch for every geometry/boundary condition.

It is obvious that the size of the matrix operators, having size $N \times N$, can be very large for moderately large values of N. As will be discussed later, the sizes of the matrices are even larger for cases with higher dimensions. Working with such large matrices in full form can be computationally demanding. Fortunately, these matrices are sparse in nature

(i.e., the number of non-zero elements is small compared to the full size of the matrix), as can be seen from the forms of D_x and D_{2x}. As such, when solving numerically, they are always defined as sparse matrices, which significantly reduces memory requirements and speeds up the operation. Almost all programming languages commonly used for numerical analysis have sparse-matrix libraries that can be used for this (e.g., Python has scipy.sparse package).

3.2. The Poisson Equation in Two Dimensions

The 2D Poisson equation in a Cartesian xy plane is given by:

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = f(x,y)\,, \quad x, y \in \Omega\,. \tag{23}$$

Here the solution $u = u(x,y)$ spans a 2D space. The matrix analysis for the 1D Poisson equation discussed in Section 3.1 can be extended for this 2D case. The matrix operators developed for the 1D case can be used as the building blocks. The key idea is to convert the 2D problem into an equivalent 1D problem. This is done by mapping the 2D solution grid into a 1D vector, solving the 1D vector, and then remapping it to the original 2D space. The main tasks are developing the mapping algorithm, figuring out how the boundary conditions are mapped from 2D to 1D space, and assembling the corresponding matrix operators.

The 2D solution space in the xy-Cartesian plane is defined as $\Omega = [x_1, x_{N_x}] \times [y_1, y_{N_y}]$. This denotes a continuous rectangular region with x values spanning from x_1 to x_{N_x} and y values spanning from y_1 to y_{N_y}. Note that the analysis would hold for any shape of solution space. In discrete space, a grid of equally spaced $N_y \times N_x$ points (y_k, x_j) are considered, where $j = 1, 2, \cdots, N_x$ and $k = 1, 2, \cdots, N_y$. The x and y spacing are denoted Δx and Δy, respectively. These points can be imagined as elements of a matrix with N_y rows and N_x columns, as shown in Figure 1. Any point can be addressed by the matrix coordinate (k, j), where k refers to the row number and j refers to the column number. The solution u at point (y_k, x_j) is denoted as $u(k, j) = u_{kj}$. The order of the indices j and k is selected in this way to be consistent with syntax of common numerical coding languages (e.g., Python and MATLAB).

Now, u_{kj}, being 2D, is not an ideal input for a matrix operator. The elements of the 2D grid are rearranged by stacking the columns on top of each other to form one large $N_x N_y \times 1$ vector, referred to as $\mathbf{u}_{1D}$. This is shown in Figure 1 [17]. Each element of this $\mathbf{u}_{1D}$ corresponds to a specific grid point (y_k, x_j) and the corresponding solution u_{kj}. The elements of $\mathbf{u}_{1D}$ can be addressed by a single index $i = 1, 2, \cdots, N_x N_y$. Note that there are many other valid approaches to mapping a 2D matrix into an 1D vector. This discussion is limited to the column-stacking method, which is relatively simple and intuitive. For this mapping, the 2D and 1D indices are related as:

$$i = N_y(j-1) + k \tag{24}$$

The opposite transformation is:

$$k = i \bmod N_y\,, \tag{25}$$

$$j = \frac{i-k}{N_y} + 1\,. \tag{26}$$

Here mod refers to the modulo operation. Using these relationships, it is possible to transfer back and forth between the 2D representation and the equivalent 1D representation. This process is referred to as *mapping*. The mapping process is shown in Figure 1.

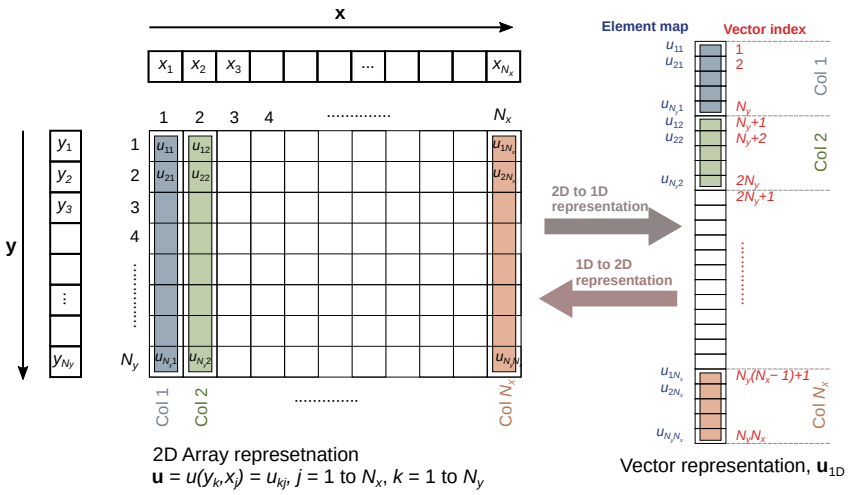


Figure 1. Process of mapping a 2D grid to a 1D vector and vice-versa.

After obtaining $\mathbf{u}_{1D}$ from the mapping process, the corresponding derivative matrix operators need to formulated. The matrix operators for the first and second partial derivatives with respect to x are expressed as D_x^{2D} and D_{2x}^{2D}, respectively. Similar quantities with respect to y are expressed as D_y^{2D} and D_{2y}^{2D}, respectively. The superscripts denote that these are 2D matrix operators. For the 1D case discussed in Section 3.1, the consecutive elements of the vector $\mathbf{u}$ corresponded to consecutive points along a 1D spatial axis. For the 2D case, however, the relation between the elements of $\mathbf{u}_{1D}$ and the corresponding spatial points is more complex. Every N_y block of elements in $\mathbf{u}_{1D}$ (shown in different colors in Figure 1) represents a column in the 2D representation. They have consecutive y values for a given x value. Within this block, matrix operators of similar form as Equation (13) and (14) would work for the y derivative. As one moves from one of these blocks to the next (moving from one color to another in Figure 1), the y values restart from the beginning and the x values increment by one step. Our x derivative operator must operate on this jumbled space and accurately pick out the correct elements for a differentiation operation.

First, the y derivatives are considered, as they are simpler to understand than the x derivatives for this mapping process. The y partial derivatives in vector form are denoted $\mathbf{u}_{y,1D}$ and $\mathbf{u}_{2y,1D}$. The 2D matrix operator for the y partial derivative must be such that it sifts each N_y block of elements and multiplies it by the 1D-derivative matrix operator. As it goes through each of the columns, the results should be stacked on a column to maintain the same spatial mapping as $\mathbf{u}_{1D}$. This can be done by forming a *block diagonal matrix* whose diagonal blocks are copies of the 1D D_y matrix operator [8,17] (D_y has the same form as D_x from Equation (13), with Δx replaced by Δy). The process is shown graphically in Figure 2. Each rectangular block in D_y^{2D} is an $N_y \times N_y$ matrix. The zero blocks are zero matrices of the same size. Each block operates on an $N_y \times 1$ sized block of $\mathbf{u}_{1D}$ (represented as different-colored columns and labeled $Col_1, Col_2, \cdots, Col_{N_x}$). This is shown in the figure by connecting a D_y^{2D} matrix block with the corresponding $\mathbf{u}_{1D}$ column block by curved lines. For a given block-row (the rows drawn in the figure with black lines) of the matrix, all elements of $\mathbf{u}_{1D}$ beside a specific block of N_y consecutive elements are ignored (i.e., multiplied by zeros). Thus, the derivative operation is performed for each of these column blocks as one moves from one block-row of the matrix to another. This matrix multiplication produces a column vector representing the y derivatives mapped in the same format as $\mathbf{u}_{1D}$. The second-derivative matrix, D_{2y}^{2D}, can be similarly constructed by replacing D_y with D_{2y} in each block.

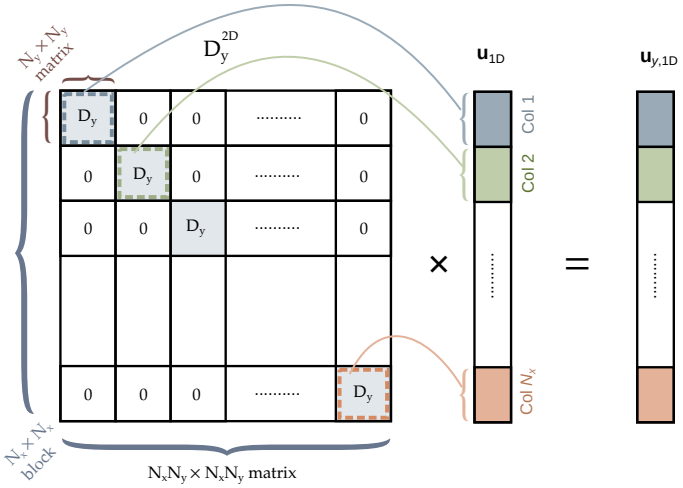


Figure 2. Matrix first-derivative operator (with respect to y) for the two-dimensional case. D_y has the same form as D_x which is defined in Equation (13).

Let us now consider the matrix operators for the x partial derivatives, D_x^{2D} and D_{2x}^{2D}. When multiplied with $\mathbf{u}_{1D}$, these matrices produce the partial derivatives in vector form denoted $\mathbf{u}_{x,1D}$ and $\mathbf{u}_{2x,1D}$, respectively. First, the locations of the elements representing consecutive x values are identified and listed in the 1D-mapped vectors. The x values increase by one when moving from left to right in a given row (move by one column) in the 2D representation (Figure 1). In the 1D map, movement of one space along a row translates into movement by N_y elements in the corresponding 1D-mapped vector. A block matrix construction is again used to form D_x^{2D} from D_x. This time, however, each row of D_x must operate on elements of $\mathbf{u}_{1D}$ that are N_y spaces apart. This can be done by adding N_y zeros after each element in a row of D_x and then copying and arranging the rows appropriately [17]. The required configuration is shown in Figure 3. The elements of D_x are repeated in the diagonal of each block in D_x^{2D}. Thus, each block is a diagonal matrix. Each block sifts out certain elements of $\mathbf{u}_{1D}$, as shown by the curved lines in the figure. Careful observation shows that these elements are indeed the ones required to calculate the derivatives at that point. A similar block matrix can be constructed for D_{2x}^{2D} by replacing D_x elements with D_{2x} elements.

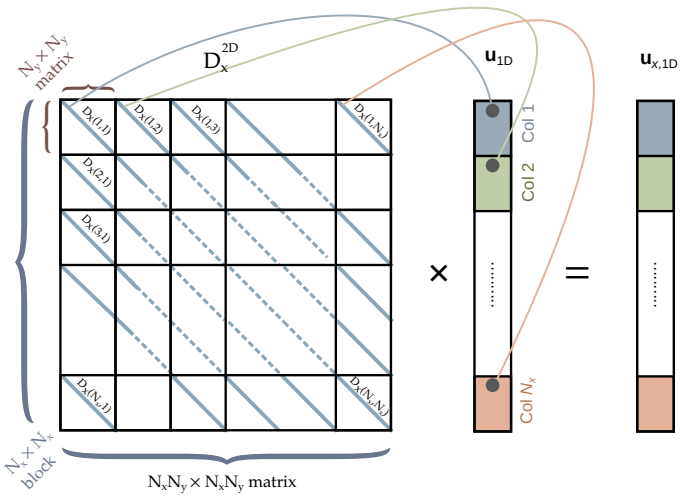


Figure 3. Matrix first-derivative operator (with respect to x) for the two-dimensional case. $D_x(j,k)$ refers to the element from row j and column k of the D_x matrix defined in Equation (13).

Now that the matrix operators have been defined, their construction using simple mathematical operations is discussed. A powerful tool in constructing block matrices is the

Kronecker product [4,18]. The Kronecker product between two matrices A (size $p \times q$) and B (size $m \times n$) is defined as:

$$A \otimes B = \begin{bmatrix} A_{11}B & A_{12}B & \cdots & A_{1q}B \\ A_{21}B & A_{22}B & \cdots & A_{2q}B \\ \vdots & \vdots & \ddots & \vdots \\ A_{p1}B & A_{p2}B & \cdots & A_{pq}B \end{bmatrix} \tag{27}$$

where A_{ij} represents the (i, j) element of matrix A, and $\otimes$ represents Kronecker product operation. Note that each term in Equation (27) is a matrix itself. The $A \otimes B$ block matrix is formed by tiling B matrices after multiplying them with an element of the A matrix. It is not difficult to see that D_y^{2D} can be constructed from the Kronecker product of D_y and an identity matrix of size N_x (denoted as I_{N_x}):

$$D_y^{2D} = I_{N_x} \otimes D_y \,. \tag{28}$$

Careful observation of Figure 3 and Equation (27) suggests that D_x^{2D} can also be computed using the Kronecker product of D_x and an identity matrix of size $N_y \times N_y$ (denoted I_{N_y}):

$$D_x^{2D} = D_x \otimes I_{N_y} \,. \tag{29}$$

The second-derivative matrices are calculated identically:

$$D_{2y}^{2D} = I_{N_x} \otimes D_{2y} \,, \tag{30}$$

$$D_{2x}^{2D} = D_{2x} \otimes I_{N_y} \,. \tag{31}$$

The overall sizes of these matrices are $N_xN_y \times N_xN_y$. This can be very large even for moderate values of N_x and N_y. However, it is noted that the sparsity of the matrices from the 1D case is maintained in the 2D case as well. Even more so than with the 1D case, any numerical implementation of the 2D Poisson equation should employ sparse-matrix algorithms.

The step after constructing the differentiation operators is formulation of the boundary operator matrix. The 2D boundary operator matrix, B^{2D}, would be identical to the one discussed in the previous section for the one-dimensional case. Equation (19) would be slightly modified to accommodate for the larger two-dimensional matrices:

$$B^{2D} = \begin{cases} I_{N_xN_y} \,, & \text{for Dirichlet boundary conditions} \\ D_x^{2D} \text{ or } D_y^{2D} & \text{for Neumann boundary conditions} \end{cases} \tag{32}$$

where $I_{N_xN_y}$ is an identity matrix of size $N_xN_y \times N_xN_y$. As there are two independent variables, there are two possible Neumann boundary conditions that can be applied. Which derivative to use for the Neumann boundary is determined by the problem statement. Having defined the boundary operator, construction of the system matrix operator is straightforward:

$$\mathcal{L} = \begin{cases} B^{2D} \,, & \text{on } \partial\Omega \\ D_{2x}^{2D} \,, & \text{otherwise} \end{cases} \,. \tag{33}$$

The right-hand vector also follows a similar definition as the 1D case:

$$\mathbf{b} = \begin{cases} \mathbf{g}_{1D} \,, & \text{on } \partial\Omega \\ \mathbf{f}_{1D} \,, & \text{otherwise} \end{cases} \,, \tag{34}$$

where $\mathbf{f}_{1D}$ and $\mathbf{g}_{1D}$ are 1D-mapped versions of the corresponding two-dimensional quantities, $\mathbf{f}(y_k, x_j)$ and $\mathbf{g}(y_k, x_j)$, respectively. Obviously, the algorithm used for the 1D mapping

here should be identical to the one used for calculating $\mathbf{u}_{1D}$. Finally, the system equation is constructed as:

$$\mathcal{L}\mathbf{u}_{1D} = \mathbf{b}\,. \tag{35}$$

Just like the one-dimensional case, this is a simple linear algebraic equation that can be solved easily. It should be noted that the solution is in a 1D-mapped vector form. Thus, an additional step of converting it back to the 2D form is necessary. This can be done using Equations (25) and (26).

It should be noted that only the B^{2D} matrix, the $\mathbf{g}_{1D}$ vector, and the construction of $\mathcal{L}$ and $\mathbf{b}$ are geometry and boundary condition dependent. The rest of the matrices and vectors are problem independent. Thus, configuring the solver for different geometries and boundary conditions is straightforward.

3.3. Extending to Three Dimensions

In the previous subsection, it was discussed how the two-dimensional system can be constructed using the one-dimensional operators as building blocks. A similar approach can be used to construct a three-dimensional system from the two-dimensional matrix operators. The process will involve Kronecker products of the identity matrices and the two-dimensional matrix operators. As this is intended to be an introductory paper, the details of the three-dimensional case are omitted for the sake of brevity.

4. Python Implementation

Python code for solving the Poisson equation in two-dimensions has been developed. It is publicly available on GitHub [13]. The code uses the previously discussed approach of constructing the matrix operators. The code is general and can solve any system when the boundary conditions are appropriately defined. In addition to the basic numerical recipe, a few additional things had to be taken into account for computational efficiency, with the most-necessary thing being the use of sparse matrices. The sparse matrix sub-package of SciPy (Scientific Python) [19] is used for constructing matrix operators. In this way, only the non-zero elements of the matrices are indexed and stored. This makes it possible to store large sparse matrices without wasting computer memory on storing the zeros. Further, mathematical operations can be performed more efficiently. For this work, the sparse linear algebra solver of SciPy, *spsolve*, was used.

In the following paragraphs, how the Python code should be used for specific problems is discussed. The focus is mainly on how geometric features and boundary conditions are defined inside the code. A few key features of the code are also highlighted.

The code requires the user to define the source function and the geometry of the problem. From those, the code automatically creates the boundary operator matrices, the system matrix, and the right-hand vectors. How the geometry is defined inside the code is discussed first. Two arrays that denote the x and y independent variables are declared. The range and the resolution of the variables are also defined here. An example code snippet is given below:

```
import numpy as np
# Define independent variables.
Nx = 300                          # No. of grid points along x direction
Ny = 200                          # No. of grid points along y direction
x = np.linspace(-6,6,Nx)          # x variable in 1D
y = np.linspace(-3,3,Ny)          # y variable in 1D
```

The size of the arrays determines the size of the matrix operators. Note that Δx and Δy are implicitly defined here. The 1D arrays are converted into 2D meshgrid data format, which is then unraveled into an equivalent 1D structure for ease of indexing later. The code for these operations is:

```
X,Y = np.meshgrid(x,y)            # 2D meshgrid
# 1D indexing
Xu = X.ravel()                    # Unravel 2D meshgrid to 1D array
Yu = Y.ravel()
```

After the space variables, the source function needs to be defined. For a source term having a closed-form expression, a simple for loop can be used to do this operation, as shown below:

```
# Source term for the diode example
f = np.zeros(Nx*Ny)
for m in range(Nx*Ny):
  if np.abs(Xu[m]) < 3.8 and np.abs(Yu[m])<1:
    f[m] = -np.tanh(Xu[m])/np.cosh(Xu[m])
```

This test source function is used in the semiconductor p–n junction example problem discussed later in Section 5.2. In this example, the source function is set equal an analytical functional at specific regions of space and set to zero elsewhere. Instead of an analytical function, tabulated values can also be used by utilizing an interpolation function. Using such piecewise definitions, source functions for most practical problems can be implemented.

The next step is to define outer boundary regions. The definition consists of the boundary values at the four outer walls (for 2D problems) as well as the types of boundary conditions. An example code snippet is given below:

```
# Dirichlet/Neumann boundary conditions at outer walls
uL, uR, uT, uB = 0, 0, 0, 0
ub_o = [uL, uR, uT, uB]
# Type of outer boundary conditions. 0 = Dirichlet,
# 1 = Neumann x derivative, 2 = Neumann y derivative
B_type_o = [1,1,2,2]
# Order: element 1 = left, element 2 = right, element 3 = top,
# element 4 = bottom
```

Now we discuss the non-trivial part of the geometry: the internal regions where boundary conditions are enforced. For now, the discussion is limited to rectangular regions. Two Python lists containing the high and low limits of x, y values are used to define these inner boundary regions. Then, the boundary type and boundary values are set as previously seen. The following code snippet defines two rectangular regions, each with Dirichlet boundary conditions (one with -2 and one with 2). This code snippet is from the semiconductor p–n junction diode example discussed later in Section 5.2.

```
# lower and upper limits of x defining the inner boundary region
# Format:[ [xlow1,xhigh1], [xlow2,xhigh2], .... ]
xb_i = [[-4,-3.8],[3.8,4]]
yb_i = [[-1,1],[-1,1]]

# Type of inner boundary conditions. 0 = Dirichlet,
# 1 = Neumann x derivative, 2 = Neumann y derivative
B_type_i = [0,0]
ub_i = [-2,2]                     # boundary values at inner region
```

The points that are within the rectangular region defined by the limits of x and y are automatically calculated by the code. Using this method, any number of arbitrary inner boundary regions can be defined as long as they are rectangular in shape.

For non-rectangular geometric features, the regions can be difficult to define by code algebraically. To tackle this, an image-based geometry-definition feature is implemented. The idea is that the geometric features can be extracted from a bitmap image file by code. Different colors in an image represent different regions (some of these will be boundary regions). A user can draw a bitmap image using any graphical editor. The graphic editor *GraphicsGale* [20], which is a freeware, is recommended. The pixels of the image will be treated as a grid point of the solution domain. With this approach, a user

can define any arbitrary geometry by drawing it in an image editor and then importing it in the solver. The code snippet of the image import feature is given below:

```
# Read image file
im = Image.open(im_file_name)
boundary_region_color_threshold = 10 # color value that identifies boundary
strct_map = np.flipud(np.array(im))  # flipud is done to match orientation
strct_map_u = strct_map.ravel()

regions = np.unique(strct_map_u)     # Find distinct color regions
n_regions = np.size(regions)         # Number of distinct color~regions

b_regions_indices_u = []
o_regions_indices_u = []

for m in range(n_regions):
  ind_u_temp = np.squeeze(np.where(strct_map_u==regions[m]))
  if regions[m] <= boundary_region_color_threshold:  # boundary region
    b_regions_indices_u.append(ind_u_temp)
  else:
    o_regions_indices_u.append(ind_u_temp)
```

The code reads an image file and identifies regions with distinct colors. An arbitrary color threshold value is defined. Any color value less than the threshold is considered a boundary region. The other colors are treated as normal regions where the solution is to be calculated. The indices of all the boundary (and non-boundary) regions are stored in a list. These indices correspond to the x and y variable indices.

Section 5.3 discusses an example problem using the image import feature. More details about the code can be found within the comments of the code file on the GitHub repository [13].

5. Numerical Results

Using the developed code, a few simple example problems are solved. The examples are limited to electrostatics. The electrostatic potential distribution in a two-dimensional space is determined by the Poisson equation in the following form:

$$\nabla^2 u(x,y) = f(x,y) = \frac{\rho}{\epsilon}, \text{ on } \Omega\,. \tag{36}$$

Here ρ is the charge density, ϵ is the permittivity, and u represents the electric potential or voltage. The electric field, $\mathbf{E}$, is given by the negative gradient of u:

$$\mathbf{E} = -\nabla u(x,y)\,. \tag{37}$$

Many interesting electrostatics problem involve solving the electric potential and electric field distribution using the Poisson equation. A few example cases are discussed here. As the focus is on the numerical approach rather than the physical system, simple source functions and boundary conditions are used that are not necessarily practical. Units of the variables are also ignored.

It should be noted that all three examples are solved using the same numerical solver with very little modification of the Python code. Only the definition of the geometry has to be redone. The rest of the code works without any further modifications. This is the distinct advantage of the proposed approach.

5.1. Example 1: Parallel Plate Capacitor

A parallel plate capacitor consists of two parallel conducting plates separated by an insulating region. When voltage is applied between the plates, the capacitor stores energy in the gap between the plates in the form of electric fields. Usually, the charge density is

taken to be zero inside a parallel plate capacitor (i.e., $\rho = 0$), which implies $f(x,y) = 0$. Thus, with a zero source term, the equation reduces to a Laplace equation:

$$\nabla^2 u(x,y) = 0\,, \text{ on } \Omega\,. \tag{38}$$

A parallel plate capacitor bounded within a larger simulation space with Neumann boundary conditions is simulated. This example is selected to show the versatility of the numerical solver in implementation of both inner and outer boundary conditions. The solution domain is selected to be be a rectangle bounded by $-6 \leq x \leq 6$, $-3 \leq y \leq 3$ (i.e., $\Omega \triangleq [-6,6] \times [-3,3]$). The outer edge boundaries are defined as:

$$\partial\Omega_1 \triangleq \{(x,y)\,|\,x = -6\} \text{ (left outer edge)}\,, \tag{39}$$
$$\partial\Omega_2 \triangleq \{(x,y)\,|\,x = 6\} \text{ (right outer edge)}\,, \tag{40}$$
$$\partial\Omega_3 \triangleq \{(x,y)\,|\,y = -3\} \text{ (bottom outer edge)}\,, \tag{41}$$
$$\partial\Omega_4 \triangleq \{(x,y)\,|\,y = 3\} \text{ (top outer edge)}\,. \tag{42}$$

The boundary conditions at the outer edges are taken to be:

$$\frac{\partial u}{\partial x} = 0\,, \text{ at } \partial\Omega_1 \text{ and } \partial\Omega_2\,, \tag{43}$$
$$\frac{\partial u}{\partial y} = 0\,, \text{ at } \partial\Omega_3 \text{ and } \partial\Omega_4\,. \tag{44}$$

These are Neumann boundaries that force the electric field to be zero at the outer edges. Again, these boundaries are not usually important, as only the solution within the region bounded by the parallel plates is of interest. These extra boundary conditions are added to create a more complex demonstration problem.

Next, two inner boundary regions are defined by two rectangles representing the parallel plates:

$$\partial\Omega_5 \triangleq \{(x,y)\,|\,-2 \leq x \leq 2 \text{ and } 0.5 \leq y \leq 0.6\} \text{ (top plate)} \tag{45}$$
$$\partial\Omega_6 \triangleq \{(x,y)\,|\,-2 \leq x \leq 2 \text{ and } -0.6 \leq y \leq -0.5\} \text{ (bottom plate)} \tag{46}$$

Dirichlet boundary conditions are set at these boundaries:

$$u = 2\,, \text{ at } \partial\Omega_5\,, \tag{47}$$
$$u = -2\,, \text{ at } \partial\Omega_6\,. \tag{48}$$

These values represent the voltage applied to these plates. The solution domain with all the boundary regions is shown in Figure 4.

Now that the problem, the solution domain, and the boundary conditions are defined, the process of constructing the boundary operator matrix, the system matrix, and the right-hand vector can be started. The boundary operator follows Equation (19):

$$B = \begin{cases} D_x^{2D}\,, & \text{at } \partial\Omega_1\,, \partial\Omega_2 \\ D_y^{2D}\,, & \text{at } \partial\Omega_3\,, \partial\Omega_4 \\ I_N\,, & \text{at } \partial\Omega_5\,, \partial\Omega_6 \end{cases} \tag{49}$$

The system matrix has the same form as defined in Equation (35). The right hand vector follows:

$$\mathbf{b} = \begin{cases} 2\,, & \text{on } \partial\Omega_5 \\ -2\,, & \text{on } \partial\Omega_6 \\ 0\,, & \text{otherwise} \end{cases} \tag{50}$$

With all the mathematical quantities defined, the linear system of the equation is solved using a sparse-matrix solver. After mapping the solution back into 2D space, $u(x, y)$ and $-\nabla u$ are plotted in Figure 5. From the $u(x, y)$ contour plot, it can be seen that the top and bottom electrodes have values of $+2$ and -2, respectively, and the potential gets distributed to other points in space. The $-\nabla u$ distribution shows how the electric field lines point from the positive voltage side to the negative voltage side. The lines inside the capacitor are oriented vertically, as one would expect from an ideal parallel plate capacitor. It is also noted that the fringe fields at the edge of the plates are curved. The field magnitude is large at the edges due to the sharp potential transition. The electric field lines outside the capacitor region are mostly determined by the Neumann boundary conditions enforced at the outer edges of the solution domain. As previously stated, the solution at these locations is not usually relevant when studying a parallel plate capacitor unless one wishes to study the fringe fields. It is possible to limit the solution domain to the inside of the capacitor only, which would require setting only outer-edge boundaries. The slightly more-complicated boundaries were enforced for demonstration purposes. The proposed scheme can model physical systems that require boundary conditions to be set both at the outer edges as well as at inner regions.

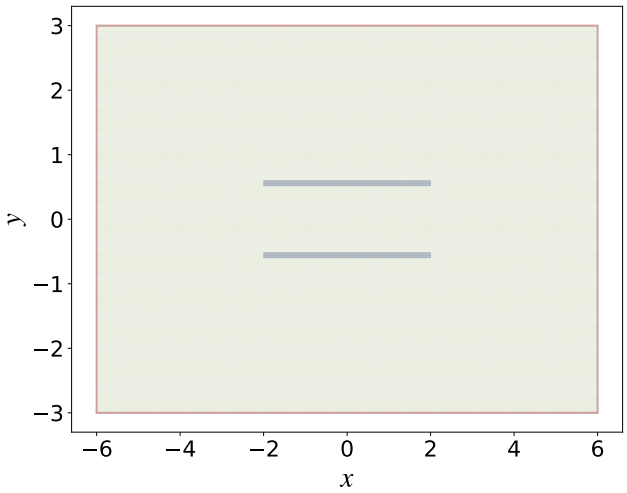
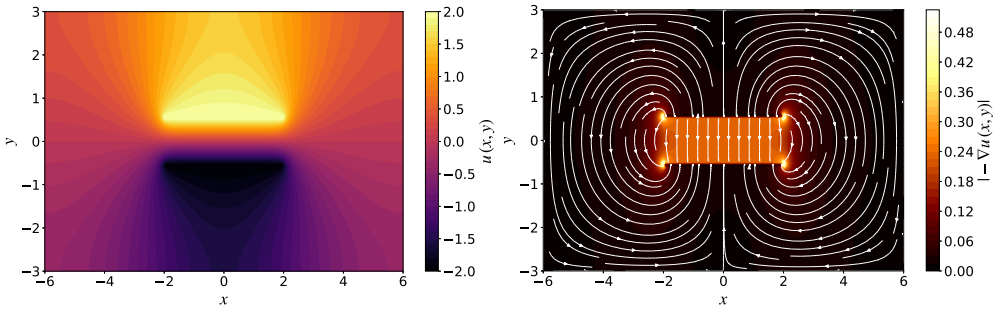


Figure 4. Solution domain for the two-dimensional parallel plate capacitor problem. The boundaries defined by the red points are Neumann boundaries. The gray regions indicate the parallel plates, where Dirichlet boundary conditions are set. The light green regions have no boundary conditions enforced on them.



(**a**) (**b**)

Figure 5. Solution of the Poisson equation for a parallel plate capacitor: (**a**) potential distribution, u, and (**b**) electric field distribution. The colors in (**b**) represent the magnitude of the electric field, $|-\nabla u|$, and the arrow lines are electric field lines showing the direction of $-\nabla u$.

5.2. Example 2: Semiconductor p–n Junction

For the second example, a highly simplified semiconductor p–n junction diode structure is considered. The diode consists of two different types of semiconductors that are sandwiched together. Voltage is applied at the two ends of the semiconductors. Such a device can act as a rectifier (allowing electric current to flow in only one direction). More details about device operation are avoided as they are not in the scope of this text. The potential profile along the junction of the device follows the Poisson equation as expressed in Equation (36). The charge profile along the junction can be modeled in various ways. A common model of charge density is [21]:

$$\rho = 2\rho_0 \operatorname{sech}\left(\frac{x}{a}\right) \tanh\left(\frac{x}{a}\right) . \tag{51}$$

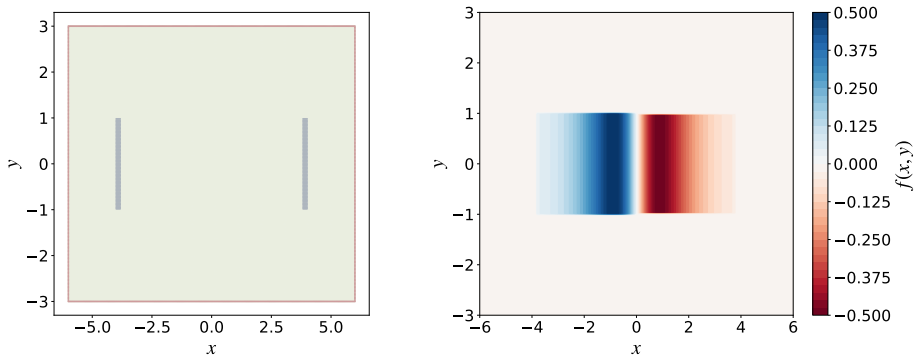
Here ρ_0 and a are parameters related to the geometrical and material properties of the device. It is assumed that the two semiconductor materials are placed along the x axis to form a junction along the line $x = 0$. Being consistent with this form of charge density profile, the source function, $f(x, y)$, is defined as:

$$f(x,y) = \begin{cases} \operatorname{sech}(x)\tanh(x)\,, & \text{for } |x| < 3.8, |y| < 1 \\ 0\,, & \text{otherwise} \end{cases} . \tag{52}$$

Here convenient values of parameters that produce a simple expression are used. Note that for this demonstration problem, only the same functional variation is maintained. For simplicity, realistic values of the functions are not used. In addition to the source function, two Dirichlet boundary regions, $\partial\Omega_5$ and $\partial\Omega_6$, are defined. These represent metal contacts on either side of the semiconductors where voltage can be applied:

$$\partial\Omega_5 \triangleq \{(x,y)\,|\,3.8 \le x \le 4 \text{ and } -1 \le y \le 1\} \text{ (right contact)} \tag{53}$$
$$\partial\Omega_6 \triangleq \{(x,y)\,|\,-4 \le x \le 3.8 \text{ and } -1 \le y \le 1\} \text{ (left contact)} \tag{54}$$

The four outer boundary edges (i.e., $\partial\Omega_1$ to $\partial\Omega_4$) have the same definitions as in the previous example (Equations (39)–(42)). The simulation domain and the source function are shown in Figure 6. As with the previous example, the region outside the device (where $f(x, y) = 0$) is not meaningful. It is simulated only as a test for the numerical solver.

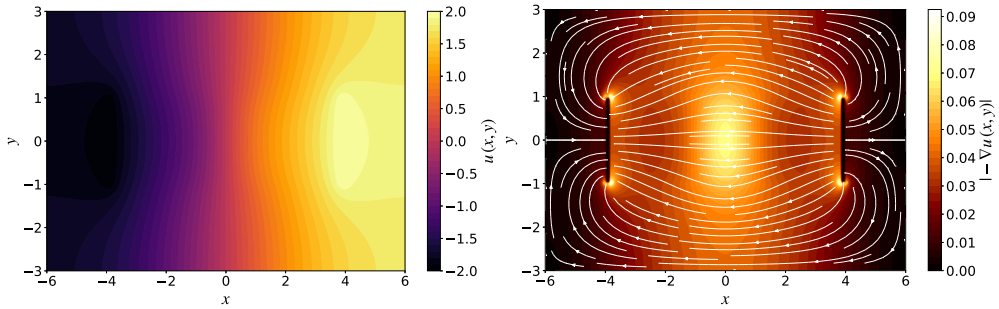


(**a**) (**b**)

Figure 6. (**a**) Solution domain for the two-dimensional p–n junction problem. The red points are Neumann boundaries. The gray regions indicate the inner Dirichlet boundaries. (**b**) The source function $f(x, y)$.

The system matrix and the right-hand vector can be constructed using the same process as discussed in Section 5.1. The exact same boundary values are used, and a voltage of $u = -2$ and $u = 2$ is applied to the left and right contact regions, respectively. Please note that the voltages applied to the metal contacts affect the charge density profile

(and hence the source function $f(x, y)$). It is assumed that the $f(x, y)$ function already contains the charge profile that would exist when the aforementioned voltage is applied. More-complex modeling of the charge distribution would be needed to study the p–n junction under a variety of voltage conditions. The results for our simplified case are shown in Figure 7. It can be noted that the electric field (i.e., $-\nabla u$) is maximum near the junction along $x = 0$, which is to be expected for this configuration [21].



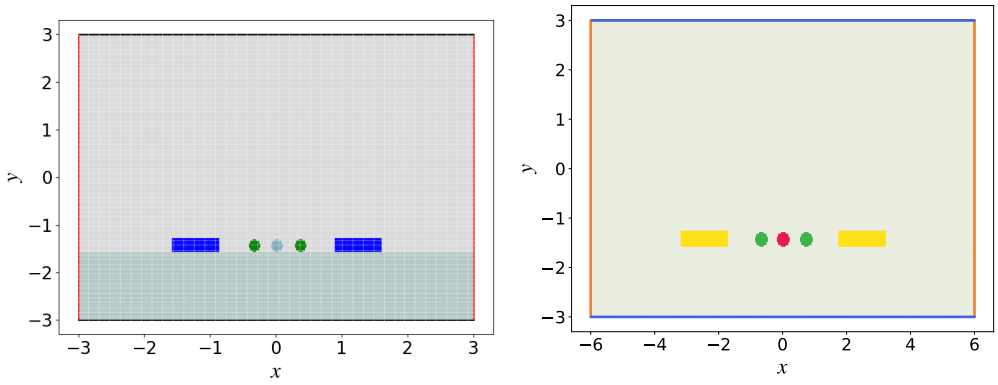
(**a**) (**b**)

Figure 7. Solution of the Poisson equation for a simplified p–n junction: (**a**) potential distribution, u, and (**b**) electric field distribution. The colors in (**b**) represent the magnitude of the electric field, $|-\nabla u|$, and the arrow lines are electric field lines showing the direction of $-\nabla u$.

5.3. Example 3: Complex Arbitrary Geometries with Image Import

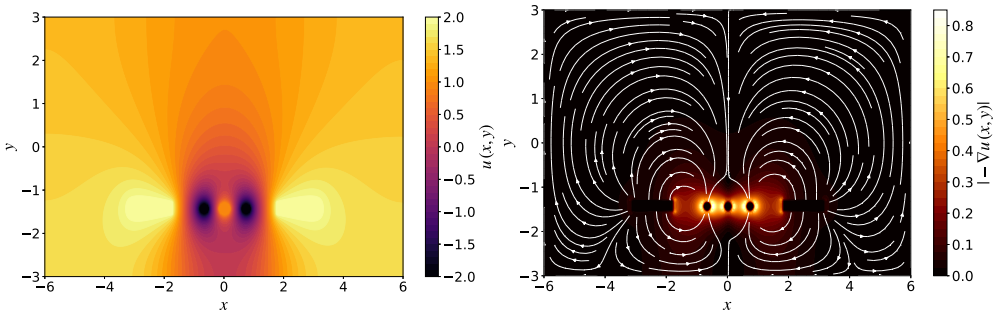
There are cases where the geometry of a problem can be more complex than that of the two examples discussed above. For those cases, it can be difficult to define the geometry through text data or code. For example, any region with non-rectangular shapes can be cumbersome to define by code alone. For such cases, the image import feature discussed in Section 4 is used. This feature is highlighted in this example. First, an arbitrary geometry is drawn in a 300 pixel $\times$ 200 pixel grid using the free image editor GraphicsGale [20]. Note that this can be done using any image editor. Each separate region of the geometry is drawn using a separate color. For this example, the geometry shown in Figure 8a is selected. The image is imported into the solver using the developed code, and the boundary regions are highlighted in Figure 8b. In the following, all colors are referred to with respect to Figure 8b. The orange and blue pixels represent zero Neumann boundary conditions (x- and y-derivative Neumann boundary condition, respectively) at the outer walls. The yellow rectangular regions are a Dirichlet boundary of value 2. The green circular regions are a Dirichlet boundary of value -2, and the red circular region is a Dirichlet boundary of value 1. Note that some colored regions of Figure 8a are not reflected in Figure 8b because they are not defined as boundary regions. Those regions are considered simple solution regions instead. Note that there can be more than one of these regions (as in this example). Having multiple such regions can help us define complex source functions that have different values in different regions. For simplicity, a source function that has zero values everywhere is assumed.

The Poisson equation is solved for the aforementioned geometry, and the results are shown in Figure 9. Like the previous examples, these results are interpreted in context of electric potential and electric field. Both the potential and the electric field have the expected distribution.



(**a**) (**b**)

Figure 8. (**a**) Solution domain for the two-dimensional arbitrary problem with external-image-defined geometry. (**b**) The boundary regions identified by the code. Different colors represent different boundary conditions to be enforced.



(**a**) (**b**)

Figure 9. Solution of the Poisson equation for an arbitrary image-defined geometry: (**a**) potential distribution, u, and (**b**) electric field distribution. The colors in (**b**) represent the magnitude of the electric field, $|-\nabla u|$, and the arrow lines are electric field lines showing the direction of $-\nabla u$.

6. Conclusions

A method for solving the Poisson equation in 1D and 2D using a finite difference approach is presented. The main focus was on the systematic construction of the solver so that it can be applied to a variety of problems involving the Poisson equation. First, the matrix operators for the derivative operation and the matrix operators for boundary condition enforcement were constructed. The two matrices were combined to form a system matrix operator. A similar operation was performed for the right-hand side vector. Thus, a systematic approach of converting the Poisson equation into a linear algebraic/matrix equation was developed. The boundary conditions with geometry information were coded into a matrix separately from the derivative operator, making the approach easy to adjust for different geometries and boundary conditions. Three example problems were solved using the developed approach. The key benefit of the proposed method is that very little modifications of the numerical solver are needed to handle three problems that are significantly different from each other. The procedure for setting up the solver for these problems is discussed thoroughly. Further, the Python code for the algorithm is available to the readers though a GitHub repository. It should be noted that the proposed method has some of the drawbacks inherent with all finite difference solvers, for example, meshing constraints when handling complex geometries and boundaries.

Funding: This research received no external funding.

Data Availability Statement: The data that support the findings of this study are openly available in the GitHub repository at https://github.com/zaman13/Poisson-solver-2D, Accessed date: 1 July 2022.

Acknowledgments: The author thanks Paul C. Hansen.

Conflicts of Interest: The author declares no conflict of interest.

List of Symbols

∇^2	Laplacian operator in Cartesian coordinates
u	Dependent variable of the Poisson equation (to be solved)
f	Source function of the Poisson equation
g	Boundary function of the Poisson equation
Ω	The solution domain
$\partial\Omega$	Boundary of the solution domain
$\mathbf{u}$	Vectorized form of u on the solution grid
$\mathbf{f}$	Vectorized form of f on the solution grid
$\mathbf{g}$	Vectorized form of g on the solution boundary
$\mathbf{b}$	The right hand vector
$\mathbf{u_{1D}}$	1D-mapped vectorized form of u for the 2D problem
$\mathbf{f_{1D}}$	1D-mapped vectorized form of f for the 2D problem
$\mathbf{g_{1D}}$	1D-mapped vectorized form of g for the 2D problem
D_x	First-derivative matrix operator with respect to x in 1D
D_y	First-derivative matrix operator with respect to y in 1D
D_{2x}	Second-derivative matrix operator with respect to x in 1D
D_{2y}	Second-derivative matrix operator with respect to y in 1D
D_x^{2D}	First-derivative matrix operator with respect to x in 2D
D_y^{2D}	First-derivative matrix operator with respect to y in 2D
D_{2x}^{2D}	Second-derivative matrix operator with respect to x in 2D
D_{2y}^{2D}	Second-derivative matrix operator with respect to y in 2D
B	Boundary operator matrix
I_N	Identity matrix of dimension $N \times N$
$\mathcal{L}$	System matrix
$\otimes$	Kronecker product
$\mathbf{E}$	The electric field
ρ	Charge density
ϵ	Permittivity of the medium

References

1. Mori, Y.; Takabatake, K.; Tsugeno, Y.; Sakai, M. On artificial density treatment for the pressure Poisson equation in the DEM-CFD simulations. *Powder Technol.* **2020**, *372*, 48–58. [CrossRef]
2. Radhakrishnan, A.; Xu, M.; Shahane, S.; Vanka, S.P. A Non-Nested Multilevel Method for Meshless Solution of the Poisson Equation in Heat Transfer and Fluid Flow. *arXiv* **2021**, arXiv:2104.13758.
3. Baranyi, L. Computation of unsteady momentum and heat transfer from a fixed circular cylinder in laminar flow. *J. Comput. Appl. Mech.* **2003**, *4*, 13–25.
4. Brewer, J. Kronecker products and matrix calculus in system theory. *IEEE Trans. Circuits Syst.* **1978**, *25*, 772–781. [CrossRef]
5. Schroeder, J.; Muller, R. IGFET Analysis through numerical solution of Poisson's equation. *IEEE Trans. Electron Devices* **1968**, *15*, 954–961. [CrossRef]
6. Matsuura, T.; Saitoh, S.; Trong, D. Numerical solutions of the poisson equation. *Appl. Anal.* **2004**, *83*, 1037–1051. [CrossRef]
7. Causon, D.; Mingham, C. *Introductory Finite Difference Methods for PDEs*; Bookboon: London, UK, 2010.
8. LeVeque, R.J. *Finite Difference Methods for Ordinary and Partial Differential Equations*; Society for Industrial and Applied Mathematics: Philadelphia, PA, USA, 2007. [CrossRef]
9. Press, W.H.; Flannery, B.P. *Numerical Recipes: The Art Of Scientific Computing*, 3rd ed.; Cambridge University Press: Cambridge, UK, 2007.
10. Burden, A.M.; Burden, R.L.; Faires, J.D. *Numerical Analysis*, 10th ed.; Cengage: Boston, MA, USA, 2016. [CrossRef]

11. Nagel, J.R. Numerical Solutions to Poisson Equations Using the Finite-Difference Method [Education Column]. *IEEE Antennas Propag. Mag.* **2014**, *56*, 209–224. [CrossRef]
12. Abdallah, S. Numerical solutions for the pressure Poisson equation with Neumann boundary conditions using a non-staggered grid, I. *J. Comput. Phys.* **1987**, *70*, 182–192. [CrossRef]
13. Zaman, M. Poisson-sover-2D, 2022. Available online: https://github.com/zaman13/Poisson-solver-2D (accessed on 1 July 2022).
14. Sapa, L.; Bożek, B.; Danielewski, M. Difference Methods To One And Multidimensional Interdiffusion Models With Vegard Rule. *Math. Model. Anal.* **2019**, *24*, 276–296. [CrossRef]
15. Sapa, L.; Bożek, B.; Tkacz–Śmiech, K.; Zajusz, M.; Danielewski, M. Interdiffusion in many dimensions: mathematical models, numerical simulations and experiment. *Math. Mech. Solids* **2020**, *25*, 2178–2198. [CrossRef]
16. Dunn, S.M.; Constantinides, A.; Moghe, P.V. Finite Difference Methods, Interpolation and Integration. In *Numerical Methods in Biomedical Engineering*; Elsevier: Amsterdam, The Netherlands, 2006; pp. 163–208. [CrossRef]
17. Basilisk Software. Available online: http://basilisk.fr/sandbox/easystab/diffmat_2D.m (accessed on 12 April 2022).
18. Regalia, P.A.; Sanjit, M.K. Kronecker Products, Unitary Matrices and Signal Processing Applications. *SIAM Rev.* **1989**, *31*, 586–613. [CrossRef]
19. Virtanen, P.; Gommers, R.; Oliphant, T.E.; Haberland, M.; Reddy, T.; Cournapeau, D.; Burovski, E.; Peterson, P.; Weckesser, W.; Bright, J.; et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nat. Methods* **2020**, *17*, 261–272. [CrossRef]
20. GraphicsGale Software. Version 2.08.20. 2020. Available online: https://graphicsgale.com/us/ (accessed on 12 April 2022).
21. Hayt, W. *Engineering Electromagnetics*; McGraw-Hill: New York, NY, USA, 2012.